nanuq

MANUAL TÉCNICO

Cómo está hecho Nanuq

Arquitectura con diagramas, cada plataforma por dentro y, al final, cada función del código.





Versión 1.44.3 · 8 de noviembre de 2026 · Español · Software libre (GPL-3.0-or-later)



Índice

La parte 1 explica el diseño; la parte 2 es la referencia de cada función.

ARQUITECTURA

1 **Panorama** 6
Una sola interfaz, tres envolturas.

2 **El repositorio** 8
Qué hay en cada carpeta.

3 **La página** 9
Cómo arranca, navega y reacciona app.js.

4 **Datos y almacenamiento** 11
Lo que dice algo de ti va cifrado.

5 **Reproducción** 13
De dónde sale cada canción y qué pasa cuando algo falla.

6 **Biblioteca y Jellyfin** 14
La API que usa Nanuq y lo que arma por su cuenta.

7 **DJ Nanuq** 15
Rondas con reglas, sin IA.

8 **Mis dispositivos** 16
Un Nanuq suena y los demás lo controlan, a través de Jellyfin.

9 **La fiesta** 17
Teléfonos que mandan canciones a una computadora por la red local.

10 **La app de Android** 18
WebView + Java nativo.

11 **La app de escritorio** 19
Electron para Linux y Windows.

12 **Seguridad** 20
Lo que llega de afuera nunca se cree.

13 **Compilar y publicar** 21
Una orden para todas las plataformas.

14 **Convenciones y trampas** 22
Lo que conviene saber antes de cambiar algo.

15 **Licencia** 23
GPLv3 o posterior, archivo por archivo.

NIVEL PROFUNDO: CADA FUNCIÓN

16 **web/app.js** 26
586 funciones, en el orden del archivo.

Base: ayudantes y almacén 26
Direcciones del servidor 27
API de Jellyfin 27
Audio: formas de onda y sintetizador 29
Biblioteca demo 29
Ajustes y temas 31
Descargas (Android) 31
Estado y navegación 32
Álbumes según las etiquetas 32
Búsqueda en las letras 33
Ayudantes de dibujo 33
Vistas 33
DJ Nanuq 34
Tu año y Tu semana (video) 36
Solicitar canción (SomeDL) 38
Reproductor 38
Letras 40
Cola 41
Avisos, menús y diálogos 41
Mis dispositivos 42
Fiesta (watch party) 44
Acciones globales 46
Entrar 46

17 **Resto de la página** 47
Idiomas y service worker.

web/i18n.js 47

web/sw.js 47

18 **Android** 48
Java nativo y el puente de JavaScript.

android/app/src/main/java/com/nanuq/app/Discovery.java 48
android/app/src/main/java/com/nanuq/app/Downloads.java 48
android/app/src/main/java/com/nanuq/app/LocalServer.java 49
android/app/src/main/java/com/nanuq/app/MainActivity.java 50
android/app/src/main/java/com/nanuq/app/MediaCenter.java 51
android/app/src/main/java/com/nanuq/app/NativeHttp.java 52
android/app/src/main/java/com/nanuq/app/PlaybackService.java 52
android/app/src/main/java/com/nanuq/app/Prefetch.java 52
android/app/src/main/java/com/nanuq/app/SecureStore.java 53
android/bridge/android-bridge.js 54
android/build-apk.sh 54

19 **Escritorio** 55
Electron y su empaquetado.

linux/main.js 55
linux/preload.js 57
linux/prepare-web.js 57
linux/collect.js 57
linux/flatpak/build-flatpak.sh 57

20 **Compilación** 58
El script que compila todo.

scripts/build.sh 58

PARTE 1

Arquitectura

Cómo está armado Nanuq: una sola interfaz web, las apps que la envuelven y cómo se habla con Jellyfin, con tus otros dispositivos y con la fiesta.

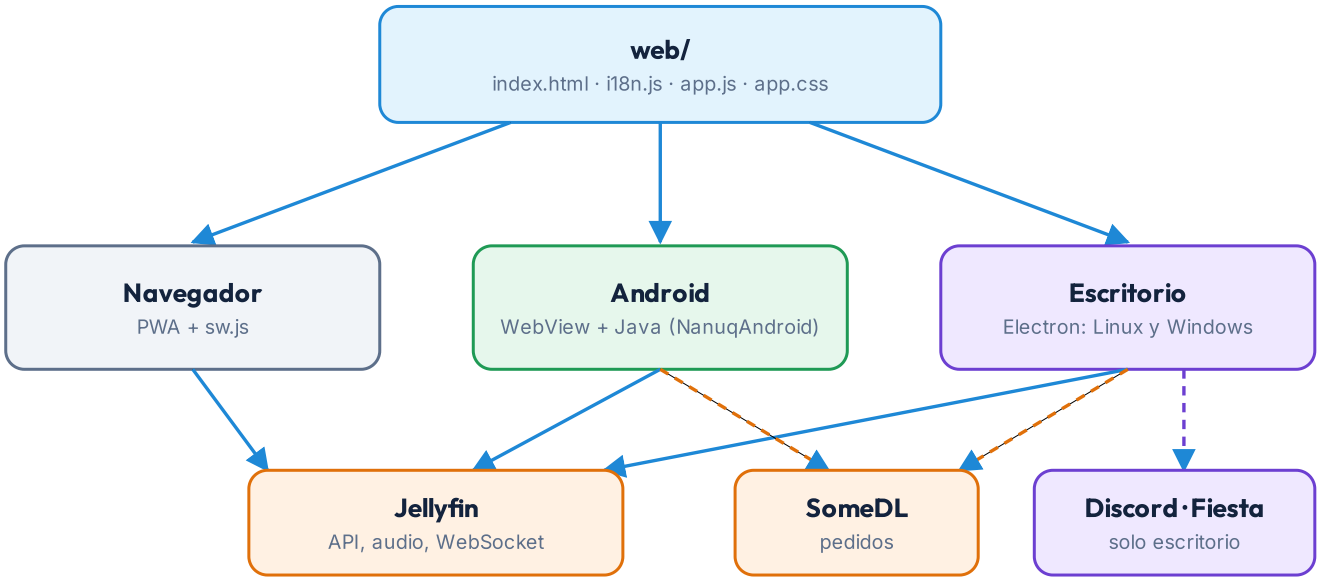


01 Panorama

Una sola interfaz, tres envolturas.



Toda la interfaz de Nanuq vive en `web/`: cuatro archivos sin compilar y sin dependencias. Las apps de Android y de escritorio no tienen interfaz propia: copian esa misma carpeta al compilar y le agregan, por un «puente», lo que una página web no puede hacer sola.



Cada plataforma sirve la misma página; las flechas punteadas son lo que solo hace el código nativo (SomeDL sin CORS, Discord, la sala de la fiesta).

Por qué así

- **Un cambio llega a todas partes**: la interfaz se toca solo en `web/`; Android y Linux la copian al compilar (`syncWeb` en Gradle, `prepare-web.js` en Electron).
- **Contexto seguro sin HTTPS**: Android sirve la página en `http://localhost` y Electron en `http://127.0.0.1:47863`. Así hay `crypto.randomUUID`, almacenamiento y service worker, y a la vez se puede hablar con un Jellyfin `http://` de la red local sin bloqueos de contenido mixto.
- **Sin servicios de terceros**: Nanuq solo habla con tu Jellyfin, tu SomeDL y (si lo activas) la app de Discord de tu computadora. Las únicas peticiones externas son las fuentes de Google Fonts.

Puertos

Puerto	Quién	Para qué
TCP 47863	Escritorio (127.0.0.1)	Sirve la página y, con la fiesta abierta, el audio de la sala (`/fiesta/<token>/…`). Puerto fijo porque el almacenamiento depende del origen.

Puerto	Quién	Para qué
TCP 47864	Escritorio (red local)	WebSocket de la fiesta (`/fiesta`) y carátulas para los invitados. Solo con la sala abierta y solo direcciones privadas.
UDP 47865	Escritorio	Responde a «Who is NanuqParty?» para que los teléfonos encuentren la sala (≤ 30 respuestas/s).
UDP 7359	Jellyfin	Búsqueda oficial de servidores («Who is JellyfinServer?»), desde Android y escritorio.
TCP aleatorio	Android (127.0.0.1)	`LocalServer` : descargas, carátulas y lo guardado por adelantado, con token en la ruta.
8096 / 8920	Jellyfin	Puertos que se asumen si escribes una IP o un nombre corto sin puerto.
5000	SomeDL	Puerto que se asume para SomeDL sin puerto.

02 El repositorio

Qué hay en cada carpeta.



Ruta	Contenido
`web/index.html`	Estructura, CSP estricta y `<defs>` SVG: íconos `#i-*` y el oso vectorizado (`#djb-*`).
`web/i18n.js`	Idiomas: `LANGS` , `LANG` , `LOC` , el diccionario `EN` (clave = texto en español) y `tx()` . Se carga antes que app.js.
`web/app.js`	Toda la lógica (5939 líneas). `const VERSION` es la fuente de verdad de la versión.
`web/app.css`	Todo el estilo, con variables por tema (18 temas).
`web/sw.js`	Service worker: guarda solo la app (red primero para el código), nunca música ni API.
`android/`	WebView + Java (`com.nanuq.app`), el puente `bridge/android-bridge.js` y `build-apk.sh` .
`linux/`	Electron: `main.js` , `preload.js` , `prepare-web.js` , `collect.js` y `flatpak/` . También arma el instalador de Windows.
`scripts/build.sh`	Sube la versión y compila todo a `dist/<versión>/` .
`docs/`	Imágenes del README y estos manuales (`docs/manuals/src` : `node docs/manuals/src/build.js`).



Ojo

`dist/` , `keystore.properties` y los `*.jks` no van al repositorio. La llave de firma del APK vive fuera (`~/.android/nanuq-release.jks`): sin ella no se pueden publicar actualizaciones.

03

La página

Cómo arranca, navega y reacciona app.js.



Arranque

1. Al cargar, `i18n.js` decide el idioma y `txStatic()` traduce los textos fijos del HTML.
2. `SEC` detecta el almacén cifrado nativo y `secMigrate()` mueve allí lo que versiones viejas dejaron en claro.
3. El bloque de arranque dibuja la barra lateral y las pestañas, aplica el tema e inicia las descargas (`DL.init`).
4. `Vault.init()` reintenta unos segundos si el Keystore o el llavero aún no responden; luego `Vault.load()` lee la sesión.
5. Con sesión: `startApp(new Jellyfin(sess))` recupera la cola y la posición (`QPOS`), restaura el DJ y abre Inicio. Sin sesión: `showLogin()`.

Navegación

`S.stack` es la pila de pantallas. `go(nombre, params, opts)` empuja una pantalla (las de `TOP` vacían la pila) y llama a `render()`, que ejecuta `V[nombre](caja, params)`. Cada vista es una función asíncrona que llena su caja; si deja `_cleanup`, se llama al salir. `back()` cierra primero lo que esté encima (menú, editor de letra, reproductor) y luego retrocede. Con la fiesta abierta (`party-host` / `in-party`) solo existe la pantalla de la fiesta.

Eventos

La CSP prohíbe scripts en línea, así que no hay `onclick=`: todo botón lleva `data-act="…"` y un único manejador de clic global decide qué hacer (sección «Acciones globales»). Las imágenes que deben desaparecer si fallan llevan `data-rm`.

Idiomas

Todo texto visible se escribe en español dentro de `tx('…')`; `i18n.js` tiene la traducción con el texto en español como clave. Con valores: `tx('… {0} …', [x])`. Cambiar de idioma guarda la posición y recarga la página, así también se traducen los textos que se arman al cargar.

Secciones de app.js

Sección	Desde la línea
Base: ayudantes y almacén	1

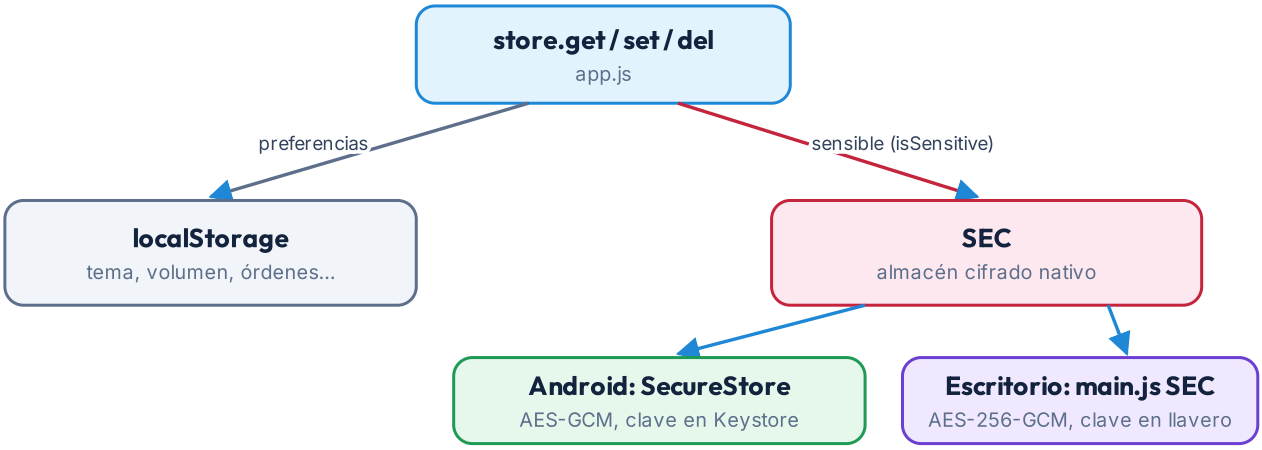
Sección	Desde la línea
Direcciones del servidor	122
API de Jellyfin	177
Audio: formas de onda y sintetizador	490
Biblioteca demo	551
Ajustes y temas	749
Descargas (Android)	780
Estado y navegación	933
Álbumes según las etiquetas	983
Búsqueda en las letras	1088
Ayudantes de dibujo	1186
Vistas	1216
DJ Nanuq	1283
Tu año y Tu semana (video)	2207
Solicitar canción (SomeDL)	2923
Reproductor	3214
Letras	3702
Cola	4048
Avisos, menús y diálogos	4119
Mis dispositivos	4130
Fiesta (watch party)	4828
Acciones globales	5572
Entrar	5732
Arranque	5912

04 Datos y almacenamiento

Lo que dice algo de ti va cifrado.



`store` es el único acceso al almacenamiento. Las preferencias de la interfaz (tema, volumen, órdenes) van a `localStorage` con el prefijo `nanuq.`; lo que dice algo de ti va al almacén cifrado de la plataforma.



En el navegador no hay almacén seguro: todo queda en localStorage (Ajustes lo dice).

Clave	Qué guarda
session	Servidor, usuario y token de Jellyfin.
queue · queuePos	Tu cola (hasta 3000) y el segundo donde ibas (queuePos no es sensible: solo un id al azar y segundos).
libIdx · lyrIdx	Índices de tu biblioteca (álbumes por etiquetas) y de tus letras.
dj · djHist	Sesión del DJ en curso e historial de lo que puso.
plays · mp3ids	Reproducciones por canción en este dispositivo y canciones que van en MP3.
somedl · somedlReqs · somedlLast	Servidor SomeDL y tus pedidos.
servers · lastServer · device · party	Servidores recientes, id del dispositivo y la fiesta en curso (teléfono).
lrc.<id>	Letras guardadas solo en este dispositivo.

Estados del almacén

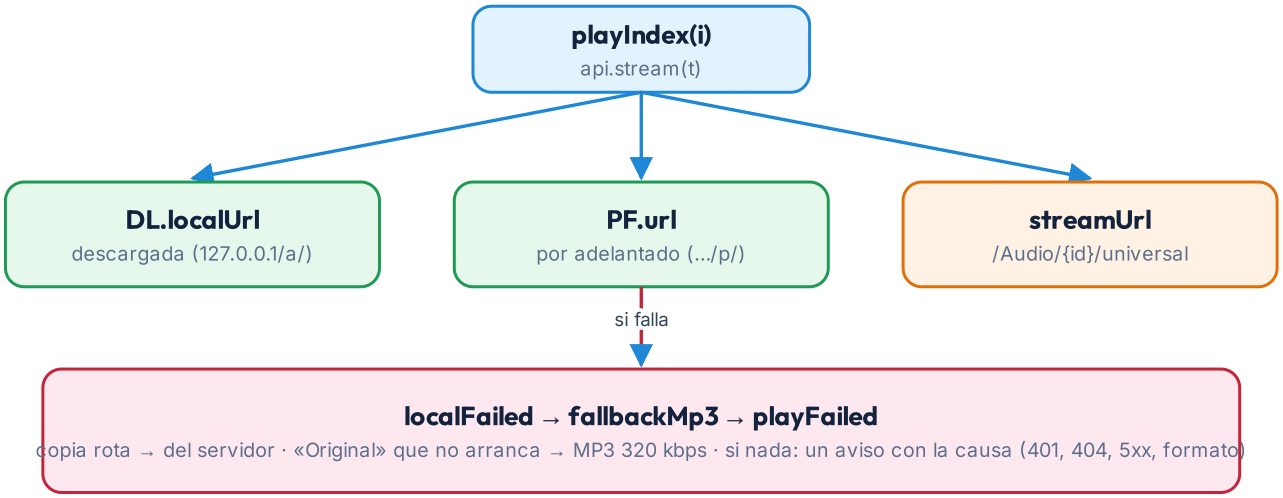
`SEC.kind` puede ser el nombre del almacén (funciona), `locked` (hay datos cifrados que ahora no abren: se reintenta), `lost` (la clave ya no existe: se ofrece «Empezar de cero», que aparta los datos sin borrarlos) o `null` (no hay almacén). Mientras no esté abierto, lo nuevo vive solo en memoria (`SEC.mem`): **nunca** se escribe en claro.

05 Reproducción

De dónde sale cada canción y qué pasa cuando algo falla.



`P` guarda la cola, el índice y el único `<audio>`. `playIndex(i)` elige la fuente con `api.stream()`: primero la copia descargada, luego la guardada por adelantado y, si no hay, el streaming de Jellyfin.



En Android, las dos primeras fuentes las sirve el teléfono mismo; en Android y escritorio la página nunca baja el audio completo a memoria.

Detalles que importan

- **Saltar en conversiones en vivo**: si Jellyfin transcodifica, el archivo no admite saltos; `seekTo` pide el audio otra vez con `StartTimeTicks` y recuerda el desplazamiento (`P.off`). Al arrastrar, la petición sale al soltar.
- **Atascos**: `armStallWatch` avisa si no arranca en 10 s; `stallCheck` reanuda desde el mismo segundo si la red se cayó a media canción; `netBack` lo hace al volver la red.
- **Desvanecer**: `pauseSoft` / `playSoft`; el volumen de vuelta es siempre el tuyo (`userVol`), nunca el del instante.
- **Pausa ajena**: si otra app tomó la salida de audio (`P.extPause`), al reanudar se recarga la canción para no quedarse mudo.
- **Controles del sistema**: `navigator.mediaSession`; en Android lo reemplaza el puente nativo. En escritorio un audio en silencio (`rcKeep` / `sessionHold`) mantiene los controles mientras carga o controlas otro dispositivo.
- **Favoritas**: Jellyfin 12 sobrescribe la favorita de la canción que suena con cada aviso de reproducción; `FAVHOLD` la vuelve a guardar después de cada aviso.

06

Biblioteca y Jellyfin

La API que usa Nanuq y lo que arma por su cuenta.



Ruta de Jellyfin	Para qué
/System/Info/Public · /Users/AuthenticateByName	Reconocer el servidor y entrar.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Listas de la biblioteca (con varias rutas de respaldo según la versión).
/Audio/{id}/universal	El audio (streaming, descargas, por adelantado, fiesta).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Leer, subir y buscar letras (10.9+).
/UserFavoriteItems/{id}	Favoritas (y la ruta anterior a 10.9 como respaldo).
/Items/{id}/InstantMix	Radio de una canción.
/Sessions · /Sessions/{id}/Playing · /socket	Mis dispositivos: sesiones, órdenes y avisos en vivo.
/Sessions/Playing(/Progress|/Stopped)	Avisos de reproducción (lo que ve el servidor).
/DisplayPreferences/nanuq-dj	Estado del DJ compartido entre tus dispositivos.
/user_usage_stats/...	Plugin Playback Reporting para Tu semana / Tu año (si está).

Álbumes por etiquetas (LIB)

Jellyfin arma los álbumes por carpeta; Nanuq los arma con las etiquetas «Álbum» + «Artista del álbum» de cada canción (`albumKeyOf`), junta «(Disc 2)» con su disco y quita duplicados. El índice de todas tus canciones se pide de a 1000 (tres páginas a la vez), se guarda cifrado y se renueva cada 10 minutos. Los ids de estos álbumes empiezan con `m` (`albumGid`).

Búsqueda en las letras (LYRX)

Jellyfin no busca dentro de las letras: Nanuq indexa las canciones con `HasLyrics` (de a 3, hasta 8000) y las letras locales, normalizadas sin acentos ni signos (`lyrNorm`). Se busca con 4 caracteres o más.

07 DJ Nanuq

Rondas con reglas, sin IA.



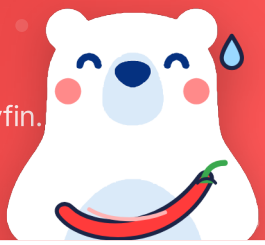
`DJ.next()` ordena los tipos de ronda al azar según su peso, evita repetir los recientes y prueba hasta encontrar una que **explore** (canciones que no sonaron en las últimas semanas). Cada canción lleva su ronda en el id de la cola: `dj-<sesión>-<ronda>-<azar>`.

Ronda	Peso	Qué pone
genre	6	Un género de tu biblioteca (≥ 3 canciones en una muestra de 400), sin repetir los recientes.
decade	3.5	Una década real de tu biblioteca; las pasadas pesan ×1.5.
discover	3	Nunca escuchadas, primero en tus géneros favoritos.
forgotten	3	Más de 45 días sin sonar.
mood	1	Según la hora (mañana tranquila, tarde con energía…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Recién llegadas, favoritas, lo de estos días y lo más escuchado (con enfriamiento).

`DJ.rank()` decide qué tan «gastada» está una canción: lo de los últimos 3 días o de sesiones recientes del DJ casi no entra; lo olvidado va primero. Al pasar la música a otro dispositivo, `DJSHARE` deja el estado en tu Jellyfin y el otro lo adopta (`DJ.adopt`); lo que se lee se limpia con `djClean`.

08 Mis dispositivos

Un Nanuq suena y los demás lo controlan, a través de Jellyfin.





No se abre ningún puerto en los dispositivos: todo pasa por tu Jellyfin, con tu sesión.

- **Quién aparece**: sesiones con cliente «Nanuq», de tu mismo usuario, activas hace menos de 15 minutos según el reloj del *servidor* (los relojes desfasados no estorban).
- **Misma red**: misma /24 en IPv4 o /64 en IPv6 (o ambas privadas), con la dirección que ve el servidor.
- **Órdenes aceptadas**: `Play` y `GeneralCommand` solo si las manda tu usuario (dato que pone Jellyfin); `Playstate` lo filtra Jellyfin por permisos.
- **Controlando**, tu cola local se aparta (`RC.localQ`) y vuelve al desconectarte; la posición se estima desde el último reporte (`RC.anchor`).
- En Android, con la pantalla bloqueada, la app nativa llama a `__nanuqTick` cada 20 s para mantener vivo el WebSocket.

09

La fiesta

Teléfonos que mandan canciones a una computadora por la red local.





UDP :47865 «Who is NanuqParty?» → nombre y puerto de la sala

El teléfono baja la canción de SU Jellyfin y se la pasa a la computadora en trozos de 256 KB, sin tenerla entera en memoria.

Mensaje	Sentido	Qué es
hello {code|key, name, theme}	teléfono → PC	Entrar con el código de 6 dígitos o volver con su llave.
welcome {you, key, room, chunk}	PC → teléfono	Entró: su llave para volver sin código (2 h).
add · [id + bytes] · end · remove	teléfono → PC	Mandar una canción (datos, letra, carátula) y sus bytes.
addOk · addErr · ready	PC → teléfono	Respuesta al pedido y aviso de que ya está lista.
state · tick	PC → teléfonos	Lista, invitados y lo que suena; segundo y línea del karaoke.
bye {why} · leave · ping	ambos	Salir, ser sacado o avisar que sigue ahí.

Topes y defensas

`PT_MAX` en `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Solo direcciones privadas (`ptLocal`), ≤ 4 conexiones sin presentarse por dirección, 5 códigos malos por minuto por dirección y, con 30 fallos en total, código nuevo y un minuto cerrado. El JSON se lee con `jparse` y los textos con `ptStr`; un mensaje que haga fallar algo solo corta esa conexión.

10 La app de Android

WebView + Java nativo.

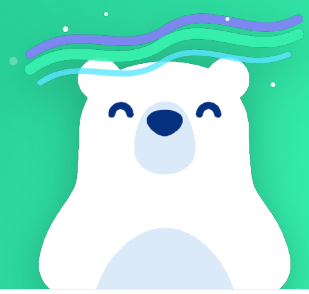


Clase	Responsabilidad
MainActivity	WebView (assets en `http://localhost`), el puente `NanuqAndroid`, latido de 20 s, recrear el WebView si muere, enlaces externos solo http(s).
MediaCenter · PlaybackService	MediaSession y notificación; servicio en primer plano con bloqueos de CPU y Wi-Fi mientras suena; se libera tras 10 min en pausa.
Downloads	Descargas de una en una, solo Wi-Fi por omisión, límite con reemplazo de automáticas; índice cifrado.
Prefetch	Lo que sigue en la cola, en la caché (≤ 400 MB por archivo, 1 GB total, 300 MB libres).
LocalServer	HTTP en 127.0.0.1 con token: `a/` audio, `c/` carátula, `p/` por adelantado; Range; 10 s por petición, ≤ 100 encabezados, 32 hilos.
SecureStore	Almacén AES-GCM con clave en Android Keystore; estados locked / lost.
NativeHttp · Discovery	HTTP sin CORS solo para las rutas de SomeDL; búsqueda UDP de Jellyfin y de fiestas.

El puente `android-bridge.js` se inyecta en `index.html` al compilar y reemplaza `navigator.mediaSession` por uno que habla con `MediaCenter`. Solo se descargan direcciones con forma de Jellyfin (`Downloads.okUrl`) y las redirecciones solo dentro del mismo servidor (`openSameHost`).

11 La app de escritorio

Electron para Linux y Windows.



Canal IPC	Qué hace
nanuq:discover	Busca servidores Jellyfin por UDP.
nanuq:http	HTTP sin CORS, solo rutas de SomeDL, sin redirecciones, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Almacén cifrado: clave AES-256 aleatoria guardada con `safeStorage` (llavero / DPAPI).
nanuq:discord	Rich Presence por el socket `discord-ipc-N` (solo sockets tuyos), con reintento mínimo y estado para Ajustes.
nanuq:party · nanuq:party-ev	Abrir/cerrar la sala y operarla (jump, order, played, now, tick, lyrics).

- Todo canal IPC comprueba `fromApp(e)`: solo la página de nuestro origen puede pedirlo.
- Ventana con `contextIsolation`, `sandbox` y sin Node; permisos solo notificaciones, pantalla completa y portapapeles.
- Fusibles de Electron: sin `runAsNode` ni `NODE_OPTIONS`, integridad del asar y solo cargar desde el asar.
- Las respuestas con el token en la URL no se guardan en la caché de disco (`no-store`).
- Servidor local: solo GET/HEAD, `Host` 127.0.0.1/localhost (contra DNS rebinding), rutas ≤ 512 y sin bytes nulos.

12 Seguridad

Lo que llega de afuera nunca se cree.



- **CSP** sin scripts en línea, sin `eval`, sin iframes ni `<base>`; solo atributos `style`.
- **JSON de afuera** con `jparse` (sin `toString` / `valueOf` / `__proto__`) y textos con `str()`: nunca `String(x)` sobre datos ajenos.
- **HTML**: todo dato pasa por `esc()`; los ids por `sid()` (solo letras, números y guiones) antes de ir a una ruta.
- **Letras** limpias de control y etiquetas (`cleanLyricsText`), ≤ 200 000 caracteres.
- **Puentes nativos** con lista blanca de rutas y topes de tamaño; nada de seguir redirecciones a otro servidor.
- **Datos personales** cifrados en disco; respaldo de Android desactivado (`allowBackup=false` y reglas de extracción).



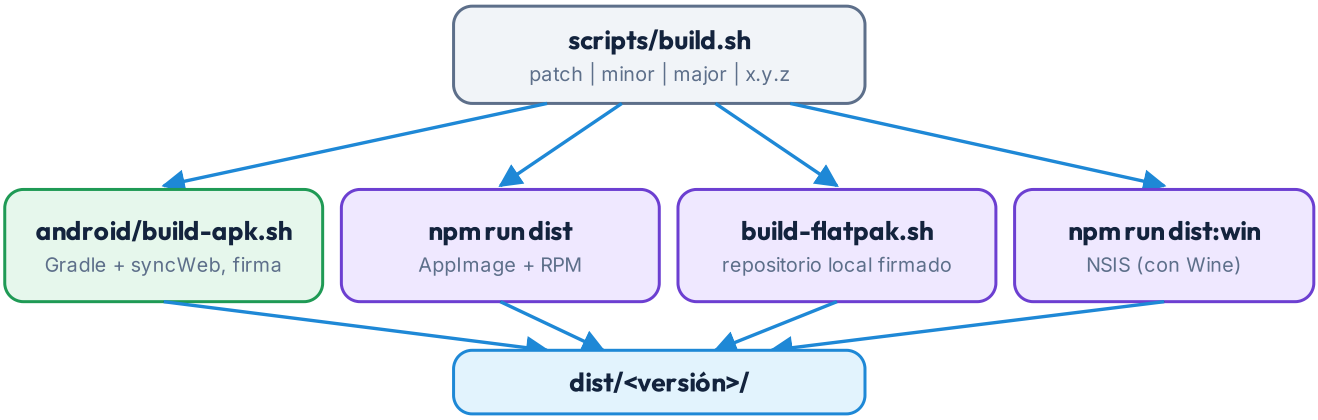
Consejo de Nanuq

El detalle completo (qué se protege y los límites conocidos) está en `SECURITY.md`.

13 Compilar y publicar

Una orden para todas las plataformas.





La versión vive en `web/app.js` (VERSION) y en `web/sw.js` (CACHE); el APK, el package.json y el Flatpak la toman de ahí.

- **Versionado**: funciones nuevas suben el número menor; arreglos, el último. El `versionCode` del APK es M·10000 + m·100 + p.
- **Android** necesita JDK 21, el SDK y `android/keystore.properties`. Firmado con la misma llave siempre.
- **Linux**: el RPM usa el `fpm` del sistema (`USE_SYSTEM_FPM=true`). El Flatpak se arma con `linux-unpacked`.
- **Comprobar**: `node --check web/app.js` y, para interfaz, Chromium en modo demo.

14 Convenciones y trampas

Lo que conviene saber antes de cambiar algo.



Regla	Por qué
Cambios de interfaz solo en `web/`	Android y Linux la copian al compilar.
Todo texto visible en `tx('…')` + su traducción en `EN`	Si no, sale en español con el idioma en inglés. Una frase completa por `tx`, sin pegar trozos.
Nada de `onclick=`, `<script>` ni `<style>` desde JS	La CSP los bloquea sin avisar.
Ids únicos en toda la app	Un id repetido escondió el cuadro de Buscar (1.30.2).
`view-transition-name` únicos	Si se repiten, la transición se cancela sin error visible.
Animaciones sin fin solo en ráfagas (`nqBurst`)	A 120 Hz cada una cuesta ≈ 1 núcleo del teléfono.
Respetar `prefers-reduced-motion`	Accesibilidad.
Tras escribir en Jellyfin, mostrar lo local y confirmar después	La lectura inmediata puede devolver lo viejo (letras, playlists, favoritas).
`.ig-crowd` sin `z-index`	Crearía su propia capa y todos quedarían detrás del DJ.

15 Licencia

GPLv3 o posterior, archivo por archivo.



Nanuq es software libre bajo la **GNU General Public License, versión 3 o (a elección de quien lo reciba) cualquier versión posterior**: `GPL-3.0-or-later`. El texto completo está en `LICENSE`, en la raíz del repositorio.

Qué significa para quien lo modifica

- Puedes usar, estudiar, copiar y modificar el código para lo que quieras.
- Si **distribuyes** Nanuq o una versión modificada (un APK, un paquete, la PWA en un servidor que otros descargan), debe ir bajo la misma licencia y con su código fuente (o una oferta para obtenerlo), con los avisos de copyright y de licencia intactos.
- Una versión modificada debe decir que lo es. Para Android, usa otro `applicationId` y tu propia llave de firma: el APK oficial solo lo firma su autor (ver «Compilar y publicar»).
- Se ofrece **sin ninguna garantía**.

Cómo está marcado el código

Cada archivo fuente empieza con dos líneas en el formato de **SPDX** (el que usan las herramientas de licencias y la especificación REUSE):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

Van como comentario del lenguaje de cada archivo (`//` , `/* */` , `<!-- -->` o `#`), después del `#!` en los scripts y del `<!DOCTYPE>` en el HTML. Al crear un archivo nuevo, agrégale las mismas dos líneas.

Dónde más aparece

Lugar	Cómo
`LICENSE`	Texto oficial de la GPLv3 (GitHub lo detecta y lo muestra en el repositorio).
README.md · README.en.md	Línea arriba, sección «Licencia» y enlace en el menú.
Ajustes de la app	Fila «Nanuq x.y.z»: la licencia y un enlace a `LICENSE` (se abre en el navegador).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
Estos manuales	Portada y este capítulo.

Lo que no es de Nanuq

Componente	Licencia	Dónde
Electron	MIT	Dentro de las apps de escritorio (Linux y Windows).
Inter · Outfit	SIL Open Font License	Fuentes, cargadas de Google Fonts (no van en el repositorio).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	El servidor: un proyecto aparte con el que Nanuq habla por su API. Nanuq no está afiliado a él.

PARTE 2

Nivel profundo: cada función

Las 841 funciones y métodos del código, archivo por archivo, con la línea donde empiezan y qué hacen.



16 web/app.js

586 funciones, en el orden del archivo.



web/app.js 5939 líneas

Base: ayudantes y almacén desde la línea 1 · 25

Función	Línea	Qué hace
$	5	Atajo de document.querySelector (opcionalmente dentro de otro elemento).
$$	6	Atajo de querySelectorAll que devuelve un arreglo.
jparse	10	Lee JSON de afuera sin toString/valueOf/__proto__.
str	11	Convierte cualquier valor a texto sin lanzar nunca un error.
esc	12	Escapa un texto para meterlo en HTML (& < > " ').
sid	16	Limpia un id del servidor: solo letras, números y guiones.
num	17	Convierte a número; si no es finito, 0.
numOrNull	18	Convierte a número o devuelve null si está vacío o no es válido.
ic	30	HTML de un ícono SVG de las <defs> (#i-nombre).
fmt	31	Formatea segundos como m:ss o h:mm:ss.
isSensitive	38	¿Esta clave guardada contiene datos personales (va al almacén cifrado)?
SEC.ok	46	¿El almacén cifrado nativo está abierto y usable?
SEC.shut	47	¿El almacén está bloqueado o perdido (no se escribe nada en claro)?
store.get	55	Lee un valor guardado (cifrado si es sensible, si no localStorage) y migra copias viejas en claro.
store.set	67	Guarda un valor; lo sensible va al almacén cifrado o, si está cerrado, solo a memoria.
store.del	78	Borra un valor guardado en el almacén y en localStorage.
secMigrate	81	Pasa al almacén cifrado lo que versiones anteriores dejaron en claro.
Vault.kind	95	Nombre del almacén cifrado en uso (o null).
Vault.init	98	Arranque en frío: reintenta abrir el almacén unos segundos antes de rendirse.
Vault.save	101	Guarda la sesión de Jellyfin.

Función	Línea	Qué hace
`Vault.load`	102	Lee la sesión guardada (o marca que el almacén está bloqueado).
`Vault.reset`	109	«Empezar de cero»: aparta los datos ilegibles y crea un almacén nuevo.
`uid`	117	Id aleatorio (UUID sin guiones).
`deviceId`	118	Id persistente de este dispositivo para Jellyfin.
`deviceName`	119	Nombre del sistema (Android, Linux, Windows…) que se anuncia a Jellyfin.

Direcciones del servidor desde la línea 122 · 1

Función	Línea	Qué hace
`parseServer`	124	Interpreta lo que escribiste como servidor (IPv4, IPv6, nombre, puerto y ruta) y arma la URL base.

API de Jellyfin desde la línea 177 · 58

Función	Línea	Qué hace
`Jellyfin.constructor`	180	Crea el cliente de Jellyfin a partir de la sesión guardada.
`Jellyfin.authHeader`	181	Arma la cabecera de autorización MediaBrowser con dispositivo, versión y token.
`Jellyfin.req`	185	Petición a la API con tiempo límite, sin caché; detecta 401 y lee el JSON con jparse.
`Jellyfin.alt`	203	Prueba varias rutas de la API hasta que una no dé 404.
`Jellyfin.probe`	204	Comprueba que una dirección sea un Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Inicia sesión con usuario y contraseña y guarda el token.
`Jellyfin.cfg`	216	Datos de la sesión para guardar.
`Jellyfin.map`	217	Convierte un elemento de Jellyfin al formato de Nanuq, limpiando ids, números y textos.
`Jellyfin.items`	236	Consulta genérica de /Items con los campos que usa la app.
`Jellyfin.albums`	240	Álbumes de Jellyfin paginados y ordenados.
`Jellyfin.item`	244	Un elemento por id (con ruta de respaldo para servidores viejos).
`Jellyfin.album`	245	Un álbum de Jellyfin y sus pistas.
`Jellyfin.artists`	249	Artistas del álbum, paginados.
`Jellyfin.artistFilters`	254	Filtros posibles para encontrar las cosas de un artista (por varios ids o por nombre).
`Jellyfin.firstNonEmpty`	260	Prueba filtros en orden y devuelve el primer resultado que no esté vacío.

Función	Línea	Qué hace
`Jellyfin.artist`	264	Un artista y sus álbumes (armados con sus canciones si hace falta).
`Jellyfin.artistTracks`	279	Todas las canciones de un artista.
`Jellyfin.genres`	287	Géneros de música probando varias rutas; si no, los saca de los álbumes.
`Jellyfin.genreParam`	306	Parámetros de consulta para uno o varios géneros (por id o por nombre).
`Jellyfin.genre`	311	Álbumes de un género.
`Jellyfin.genreTracks`	318	Canciones de un género (de las etiquetas de las canciones o de sus álbumes).
`Jellyfin.playlists`	327	Tus playlists de audio.
`Jellyfin.playlist`	328	Una playlist y sus canciones.
`Jellyfin.views`	332	Bibliotecas de música del usuario (raíz de Carpetas).
`Jellyfin.folder`	336	Contenido de una carpeta.
`Jellyfin.search`	337	Búsqueda de artistas, álbumes y canciones.
`Jellyfin.img`	344	URL de la carátula (local si está descargada en el teléfono).
`Jellyfin.streamUrl`	349	URL de streaming universal con calidad, contenedores y sesión.
`Jellyfin.stream`	354	URL para reproducir: descarga, caché por adelantado o streaming.
`Jellyfin.dlUrl`	357	URL para descargar una canción al teléfono.
`Jellyfin.peaks`	363	Baja una copia ligera y calcula la forma de onda real.
`Jellyfin.tracks`	368	Canciones de la biblioteca paginadas y ordenadas.
`Jellyfin.lyrics`	372	Letra de una canción (Jellyfin 10.9+), normalizada.
`Jellyfin.mix`	377	Canciones para una mezcla (orden, filtros, géneros, años).
`Jellyfin.uploadLyrics`	384	Sube una letra (.lrc o .txt) a Jellyfin.
`Jellyfin.searchLyrics`	391	Busca letras en los proveedores de Jellyfin (LrcLib).
`Jellyfin.pickLyrics`	395	Elige una de las letras encontradas para guardarla.
`Jellyfin.setFavorite`	397	Marca o quita una favorita.
`Jellyfin.inFavorites`	403	¿Está en la sección de favoritas de Jellyfin?
`Jellyfin.isFavorite`	404	¿Esta canción es favorita según sus datos de usuario?
`Jellyfin.favPlaylist`	411	Encuentra la playlist del plugin «Favorited Songs Playlist».
`Jellyfin.favorites`	418	Tus favoritas (filtro IsFavorite), paginadas y ordenadas.
`Jellyfin.setFavoriteLegacy`	436	Ruta antigua para guardar favoritas (segundo intento).
`Jellyfin.instantMix`	438	Mezcla instantánea de Jellyfin a partir de una canción (radio).

Función	Línea	Qué hace
`Jellyfin.addToPlaylist`	443	Agrega canciones a una playlist.
`Jellyfin.refreshLibrary`	445	Pide a Jellyfin que vuelva a escanear las bibliotecas (administradores).
`Jellyfin.savePlaylist`	446	Crea una playlist nueva con canciones.
`Jellyfin.report`	447	Avisa a Jellyfin que empezó, avanza o terminó una canción (y reafirma favoritas).
`Jellyfin.logout`	453	Cierra la sesión en el servidor (revoca el token).
`Jellyfin.wsUrl`	456	URL del WebSocket de Jellyfin.
`Jellyfin.postCapabilities`	457	Anuncia que este Nanuq acepta órdenes de control remoto.
`Jellyfin.sessions`	459	Sesiones activas (para Mis dispositivos).
`Jellyfin.sessionCmd`	460	Manda una orden de reproducción a otra sesión.
`Jellyfin.sessionPlay`	461	Manda una cola para reproducir en otra sesión.
`Jellyfin.sessionGeneral`	462	Manda una orden general (repetir, mezclar) a otra sesión.
`Jellyfin.itemsByIds`	463	Pide varios elementos por id, de 100 en 100.
`netError`	469	Mensaje claro cuando no hay conexión con el servidor.
`normLyrics`	478	Normaliza una letra de Jellyfin (sincronizada o en texto).

Audio: formas de onda y sintetizador

desde la línea 490 · 6

Función	Línea	Qué hace
`actx`	493	AudioContext compartido (creado al primer uso).
`decode`	494	Decodifica audio a un AudioBuffer.
`peaksFromBuffer`	495	Calcula los picos de la forma de onda de un AudioBuffer.
`rng`	502	Generador pseudoaleatorio con semilla.
`wav`	503	Convierte un AudioBuffer en un archivo WAV.
`synth`	510	Sintetiza una canción de prueba para el modo demo.

Biblioteca demo

desde la línea 551 · 43

Función	Línea	Qué hace
`Demo.constructor`	554	Crea la biblioteca de demostración (artistas, álbumes, canciones y playlists).
`Demo.cfg`	589	Sesión del modo demo.
`Demo.wait`	590	Simula la espera de la red.
`Demo.albums`	591	Versión demo de Jellyfin.albums.
`Demo.album`	596	Versión demo de Jellyfin.album.
`Demo.artists`	597	Versión demo de Jellyfin.artists.
`Demo.artistBy`	599	Busca un artista demo por id o por nombre.

Función	Línea	Qué hace
`Demo.artist`	600	Versión demo de Jellyfin.artist.
`Demo.artistTracks`	601	Versión demo de Jellyfin.artistTracks.
`Demo.genres`	602	Versión demo de Jellyfin.genres.
`Demo.genreTracks`	603	Versión demo de Jellyfin.genreTracks.
`Demo.genre`	604	Versión demo de Jellyfin.genre.
`Demo.playlists`	605	Versión demo de Jellyfin.playlists.
`Demo.playlist`	606	Versión demo de Jellyfin.playlist.
`Demo.views`	607	Versión demo de Jellyfin.views.
`Demo.folder`	608	Versión demo de Jellyfin.folder.
`Demo.search`	613	Versión demo de Jellyfin.search.
`Demo.img`	617	Dibuja una carátula SVG generada para el modo demo.
`Demo.buf`	631	Audio sintetizado de una canción demo (en caché).
`Demo.stream`	632	URL blob del WAV de una canción demo.
`Demo.peaks`	633	Forma de onda de una canción demo.
`Demo.tracks`	634	Versión demo de Jellyfin.tracks.
`Demo.mix`	640	Versión demo de Jellyfin.mix.
`Demo.lyrics`	650	Letra sincronizada inventada para una canción demo.
`Demo.setFavorite`	662	Versión demo de Jellyfin.setFavorite.
`Demo.isFavorite`	663	Versión demo de Jellyfin.isFavorite.
`Demo.inFavorites`	664	Versión demo de Jellyfin.inFavorites.
`Demo.favorites`	665	Versión demo de Jellyfin.favorites.
`Demo.instantMix`	666	Versión demo de Jellyfin.instantMix.
`Demo.addToPlaylist`	667	Versión demo de Jellyfin.addToPlaylist.
`Demo.savePlaylist`	668	Versión demo de Jellyfin.savePlaylist.
`Demo.report`	669	No hace nada (el demo no reporta reproducciones).
`Demo.logout`	670	No hace nada (sin servidor).
`logoInner`	675	HTML del oso del logo con el accesorio del tema.
`bearHTML`	691	HTML del Nanuq animable (logo + cara expresiva).
`bearState`	693	Estado actual del oso (bailando, contento, triste) para los osos nuevos.
`nqBurst`	699	Activa una ráfaga de animaciones por unos segundos (ahorro de batería).
`nqBurstEnd`	707	Termina la ráfaga: las animaciones se pausan y los osos quedan contentos.
`bearDance`	712	Hace bailar a todos los Nanuqs un rato.

Función	Línea	Qué hace
bearInit	717	Pone el Nanuq en los logos de la barra lateral.
bearMood	722	Cambia el ánimo de los osos al sonar o pausar la música.
showSplash	733	Pantalla de bienvenida con el Nanuq del tema.
refreshBear	744	Cambia el accesorio de los osos al cambiar de tema.

Ajustes y temas desde la línea 749 · 1

Función	Línea	Qué hace
Settings.set	776	Cambia un ajuste y lo guarda.

Descargas (Android) desde la línea 780 · 29

Función	Línea	Qué hace
DL.init	786	Carga las descargas del teléfono desde la app nativa.
DL.indexCovers	794	Índice de carátulas descargadas por id de imagen.
DL.coverUrl	798	URL local de una carátula descargada.
DL.backfillCovers	800	Completa carátulas y datos de descargas viejas o recuperadas.
DL.config	807	Manda a Android el máximo de canciones y si solo con Wi-Fi.
DL.usable	808	¿Hay descargas en este dispositivo (Android con Jellyfin)?
DL.get	809	La descarga de una canción, si existe.
DL.localUrl	810	URL local de una canción descargada y lista.
DL.isLocal	811	¿Esta URL es del servidor local del teléfono?
DL.add	813	Pide descargar una canción (manual o automática).
DL.remove	818	Quita una canción de las descargas.
DL.removeAll	819	Borra todas las descargas (o solo las automáticas).
DL.list	820	Lista de descargas.
DL.bytes	821	Espacio total que ocupan las descargas.
DL.net	822	Tipo de red actual según Android (wifi, datos, ninguna).
__nanuqDl	824	Recibe de Android los cambios de una descarga y actualiza la vista.
Plays.count	839	Veces que sonó una canción en este teléfono.
Plays.add	840	Cuenta una reproducción y guarda sola la canción a las 3.
PF.usable	857	¿Se puede guardar por adelantado (Android con Jellyfin)?
PF.isUrl	858	¿Esta URL es de lo guardado por adelantado?
PF.url	859	URL local de una canción ya guardada por adelantado (o null).

Función	Línea	Qué hace
PF.soon	860	Programa una actualización de la ventana por adelantado.
PF.sync	861	Calcula qué canciones de la cola guardar (N minutos) y se lo pide a Android.
fmtMB	876	Formatea bytes como MB o GB según el idioma.
dlStatus	877	Texto del estado de una descarga.
dlStatusShort	888	Estado corto de una descarga para el botón del reproductor.
dlSummary	894	Resumen de las descargas (cantidad, espacio, pendientes).
themeSummary	918	HTML del tema elegido en Ajustes.
applyTheme	922	Aplica un tema (colores, barra del sistema, osos).

Estado y navegación desde la línea 933 · 8

Función	Línea	Qué hace
regList	939	Registra una lista de canciones para que los botones la encuentren.
cur	948	Pantalla actual de la pila de navegación.
go	949	Navega a una pantalla (raíz o encima de la actual).
back	958	Vuelve atrás: cierra diálogos, el reproductor o la pantalla actual.
cacheDrop	968	Olvida las vistas en caché que empiezan con un prefijo.
refreshIf	970	Redibuja la pantalla actual si cumple una condición.
playlistsChanged	973	Olvida y vuelve a pedir una playlist que cambió.
cached	980	Caché de vistas con tope (las más viejas salen primero).

Álbumes según las etiquetas desde la línea 983 · 15

Función	Línea	Qué hace
albumNorm	989	Normaliza un nombre para comparar álbumes.
albumKeyOf	991	Clave del álbum de una canción según sus etiquetas (artista + álbum + disco).
albumGid	998	Id estable de un álbum armado por etiquetas.
LIB.who	1001	Identifica servidor y usuario del índice.
LIB.index	1003	Índice de álbumes por etiquetas (desde caché cifrada o del servidor).
LIB.peek	1016	El índice si ya está listo, sin esperar.
LIB.reset	1017	Marca el índice como viejo para renovarlo.
LIB.refresh	1018	Vuelve a armar el índice con todas las canciones.
LIB.fetchAll	1024	Baja todas las canciones de la biblioteca (3 páginas a la vez).
LIB.pack	1034	Versión compacta de una canción para guardar el índice.

Función	Línea	Qué hace
`LIB.unpack`	1035	Lee una canción compacta del índice guardado, limpiándola.
`LIB.save`	1040	Guarda el índice cifrado (si cabe).
`LIB.build`	1045	Agrupa las canciones en álbumes, sin repetidas.
`LIB.albumsOf`	1070	Álbumes (por etiquetas) de un grupo de canciones.
`LIB.sorted`	1075	Lista de álbumes ordenada.

Búsqueda en las letras desde la línea 1088 · 10

Función	Línea	Qué hace
`lyrNorm`	1094	Normaliza texto de letras para buscar (sin acentos ni signos).
`LYRX.load`	1097	Carga el índice de letras guardado.
`LYRX.add`	1106	Agrega la letra de una canción al índice.
`LYRX.save`	1107	Guarda el índice de letras cifrado (con espera).
`LYRX.reset`	1115	Borra el índice de letras.
`LYRX.build`	1117	Arma o completa el índice leyendo letras de a 3.
`LYRX.search`	1143	Busca canciones cuya letra contiene un texto.
`lyrSnippetHTML`	1160	Línea de letra con lo buscado resaltado (escapada).
`lyricsHitsHTML`	1165	Lista HTML de canciones encontradas por la letra.
`lyricsSection`	1170	Sección «En las letras» que se completa mientras se indexa.

Ayudantes de dibujo desde la línea 1186 · 8

Función	Línea	Qué hace
`coverHTML`	1188	HTML de una carátula con su ícono de respaldo.
`head`	1193	Encabezado de una pantalla (título, botones, oso, atrás).
`albumCard`	1197	Tarjeta de un álbum.
`artistCard`	1198	Tarjeta de un artista.
`trackRows`	1199	Contenedor de filas de canciones.
`rowsHTML`	1200	Filas de canciones (número, título, duración, menú).
`loading`	1212	Indicador de carga.
`errBox`	1213	Caja de error con «Reintentar».

Vistas desde la línea 1216 · 6

Función	Línea	Qué hace
`V.albums`	1219	Pantalla Álbumes (por etiquetas, de 120 en 120).
`V.album`	1234	Pantalla de un álbum.
`normTxt`	1265	Normaliza texto (sin acentos, minúsculas).

Función	Línea	Qué hace
mixDef	1266	Definición de una mezcla (consulta según géneros o años).
mixTracks	1280	Canciones de una mezcla (en caché).
greeting	1281	Saludo según la hora.

DJ Nanuq desde la línea 1283 · 82

Función	Línea	Qué hace
djBearSVG	1289	SVG de DJ Nanuq con audífonos, bufanda y tornamesa.
djMoment	1341	Momento del día y ánimo para la ronda «Para este momento».
pick	1342	Elige uno al azar.
wpick	1344	Elige al azar con pesos.
wshuffle	1346	Ordena al azar dando más chance a lo que pesa más.
djQid	1349	Id de cola que lleva la sesión y la ronda del DJ.
djTag	1350	Lee sesión y ronda de un id de cola del DJ.
djSame	1351	¿Dos ids son de la misma sesión del DJ?
DJ.voiceOk	1354	¿Hay una voz del sistema en el idioma de la app?
DJ.save	1355	Guarda la sesión del DJ y la comparte con tus dispositivos.
DJ.restore	1360	Recupera la sesión del DJ al abrir la app.
DJ.sample	1367	Muestra al azar de la biblioteca para elegir géneros y décadas.
DJ.heavy	1373	Tus 60 canciones más escuchadas.
DJ.pickGenre	1379	Elige el género de una ronda (sin repetir los recientes).
DJ.pickDecade	1389	Elige la década de una ronda.
DJ.rank	1401	Qué tan gastada está una canción (más bajo = va antes).
DJ.pool	1410	Canciones candidatas para un tipo de ronda.
DJ.build	1443	Arma una ronda de ~5 canciones con su presentación.
DJ.next	1470	Elige y arma la siguiente ronda (pesos y enfriamiento).
DJ.items	1484	Convierte una ronda en canciones de la cola y la anota.
DJ.lastKey	1492	Número de la última ronda.
DJ.start	1493	Empieza una sesión del DJ (aquí o en el dispositivo que controlas).
DJ.share	1519	Deja en Jellyfin el estado del DJ para otro dispositivo.
DJ.handedOff	1524	La sesión pasó a otro dispositivo: aquí se apaga.
DJ.adopt	1526	Sigue aquí una sesión que viene de otro dispositivo.
DJ.queueFromShare	1531	Reconstruye la cola del DJ desde el estado compartido.

Función	Línea	Qué hace
DJ.queueFromTags	1533	Reconstruye la cola del DJ desde los ids de la cola.
DJ.remoteTag	1535	¿Lo que suena en el dispositivo que controlas es el DJ?
DJ.shown	1536	¿Hay que mostrar el DJ?
DJ.remoteTrack	1538	Trae las rondas del DJ que suena en otro dispositivo.
DJ.stop	1548	Apaga el DJ (la cola queda como está).
DJ.onTrack	1550	Al cambiar de canción: presenta la ronda nueva y agrega la siguiente.
DJ.extend	1558	Agrega la siguiente ronda al final de la cola.
DJ.switchUp	1568	«Cambiar de onda»: otra ronda en lugar de lo que quedaba.
DJ.announce	1576	Presenta una ronda (globo, aviso y voz).
DJ.show	1578	Muestra la presentación donde corresponda.
DJ.speak	1583	Lee la presentación en voz alta y baja la música.
DJ.visible	1601	¿La página está a la vista?
DJ.bears	1602	Enciende o apaga el baile de los DJ.
DJ.say	1606	Globo de diálogo del DJ en el reproductor.
DJ.scene	1611	Escena del disco en el reproductor (funda, vinilo, entradas y salidas).
DJ.paint	1630	Actualiza todo lo del DJ en pantalla.
DJSHARE.ok	1660	¿Se puede compartir el DJ (Jellyfin y sincronización activada)?
DJSHARE.state	1661	Estado del DJ para compartir (acotado).
DJSHARE.push	1666	Guarda el estado del DJ en /DisplayPreferences.
DJSHARE.pushRaw	1674	Guarda un estado ya armado.
DJSHARE.pushSoon	1678	Guarda el estado en 1.5 s.
DJSHARE.offSoon	1680	Avisa que el DJ se apagó, si la sesión sigue siendo de aquí.
DJSHARE.pull	1684	Lee el estado compartido del DJ.
djClean	1690	Limpia el estado del DJ leído (dato no confiable).
djLiveSync	1705	Anima el DJ solo mientras se ve.
djTileHTML	1717	Tarjeta de DJ Nanuq en Inicio.
V.dj	1720	Pantalla de DJ Nanuq.
nanuqSceneSVG	1744	Escena de Nanuq para una celda vacía de Inicio.
nanuqBear	1760	Nanuq listo para colocar en una escena.
nanuqWideSVG	1764	Escena ancha de Nanuq (dos celdas).
nanuqWideTileHTML	1781	Celda ancha con escena.
nanuqTileHTML	1786	Celda con escena.

Función	Línea	Qué hace
`nanuqVisit`	1793	Un Nanuq de otro tema visita una celda.
`nanuqVisits`	1813	Programa visitas de vez en cuando.
`fillGaps`	1821	Rellena los huecos de la cuadrícula de Inicio con escenas.
`tileHTML`	1834	Mosaico de una mezcla.
`offlineBanner`	1839	Aviso «Sin conexión» con acceso a Descargas.
`V.home`	1844	Pantalla Inicio.
`V.mix`	1889	Pantalla de una mezcla.
`V.songs`	1899	Pantalla Canciones (paginada).
`V.artists`	1927	Pantalla Artistas.
`fmtLong`	1934	Duración larga («3 h 12 min»).
`V.artist`	1936	Pantalla de un artista (canciones, álbumes, aparece en).
`V.genres`	2016	Pantalla Géneros.
`V.genre`	2021	Pantalla de un género.
`V.playlists`	2032	Pantalla Playlists.
`V.playlist`	2037	Pantalla de una playlist.
`V.favorites`	2046	Pantalla Favoritas (confirmada con Jellyfin).
`V.folders`	2072	Pantalla Carpetas.
`V.search`	2081	Pantalla Buscar (incluye la búsqueda en letras).
`songsBlock`	2111	Bloque de canciones en lista o cuadrícula.
`V.queue`	2118	Pestaña Cola (teléfono).
`V.more`	2119	Pantalla Más.
`V.settings`	2122	Pantalla Ajustes.
`V.downloads`	2185	Pantalla Descargas.
`renderDownloadsSoon`	2201	Redibuja Descargas en un momento.

Tu año y Tu semana (video) desde la línea 2207 · 37

Función	Línea	Qué hace
`History.purge`	2215	Borra el historial local de versiones anteriores.
`ServerStats.available`	2225	¿Está el plugin Playback Reporting y se puede leer?
`ServerStats.plays`	2233	Reproducciones entre dos fechas desde el plugin (día por día).
`Recap.total`	2289	Duración total del video.
`Recap.setMode`	2290	Cambia entre resumen del año y de la semana.
`Recap.plays`	2295	Reproducciones del periodo (plugin, datos básicos o demo).

Función	Línea	Qué hace
`Recap.tally`	2312	Agrupa reproducciones: canciones, artistas, géneros, horas, días, racha.
`Recap.collect`	2336	Datos del resumen del año.
`Recap.collectWeek`	2348	Datos del resumen de la semana pasada (lunes a domingo).
`Recap.jellyfinYear`	2364	Canciones escuchadas en un periodo según los datos básicos de Jellyfin.
`Recap.assets`	2377	Prepara imágenes del video: osos, DJ, carátulas y paleta.
`Recap.loadImg`	2399	Carga una imagen con topes (8 MB, reducida) para el canvas.
`Recap.sceneAt`	2413	Qué escena toca en un segundo del video.
`Recap.frame`	2414	Dibuja un cuadro del video (con transición entre escenas).
`Recap.transition`	2423	Transición entre dos escenas (deslizar, círculo, zoom, barrido).
`Recap.bg`	2433	Fondo de una escena: degradado, brillo al ritmo y copos.
`Recap.weather`	2450	Dibuja el pronóstico musical (sol, tormenta, nieve, arcoíris…).
`Recap.bear`	2467	Dibuja un Nanuq que brinca al ritmo.
`Recap.text`	2474	Texto ajustado al ancho, con sombra.
`Recap.cover`	2484	Carátula con esquinas redondeadas.
`Recap.confetti`	2490	Lluvia de confeti.
`Recap.scene`	2498	Dibuja una escena del resumen.
`Recap.music`	2745	Música del video generada con Web Audio (con redoble).
`Recap.stop`	2774	Detiene la reproducción o grabación del video.
`Recap.play`	2775	Reproduce el video en el canvas y, si se pide, lo graba (MediaRecorder).
`Recap.save`	2803	Guarda el video (Android en trozos, escritorio como descarga).
`mp4Finalize`	2829	Escribe la duración real en el MP4 fragmentado.
`recapCloseAudio`	2870	Cierra el audio del video una sola vez.
`ease.out`	2871	Curva de animación (salida).
`ease.inOut`	2871	Curva de animación (entrada y salida).
`ease.back`	2871	Curva de animación con rebote.
`clamp01`	2872	Limita un valor entre 0 y 1.
`mixHex`	2873	Mezcla dos colores hex.
`hexA`	2874	Color hex con transparencia.
`recapYearOpen`	2876	¿Es la primera semana de diciembre?
`recapTileHTML`	2877	Tarjeta «Tu año con Nanuq» en Inicio.
`V.recap`	2880	Pantalla de Tu semana / Tu año (ver y exportar).

Solicitar canción (SomeDL) desde la línea 2923 · 26

Función	Línea	Qué hace
`__nanuqHttp`	2929	Recibe la respuesta de una petición nativa a SomeDL (Android).
`reqNative`	2930	¿Hay puente nativo para hablar con SomeDL sin CORS?
`reqRawHttp`	2931	Petición a SomeDL por el puente nativo o fetch.
`parseSomedl`	2941	Interpreta la dirección de SomeDL (puerto 5000 por omisión).
`reqNetError`	2950	Mensaje claro cuando SomeDL no responde.
`REQ.save`	2965	Guarda la lista de pedidos.
`REQ.setCfg`	2966	Guarda o quita la dirección de SomeDL.
`REQ.pending`	2967	Pedidos que siguen en camino.
`REQ.visible`	2968	¿La app está a la vista (para revisar más seguido)?
`REQ.http`	2969	Petición a la API de SomeDL con manejo de errores.
`REQ.probe`	2977	Comprueba que una dirección sea SomeDL.
`REQ.add`	2983	Manda un pedido de canción a SomeDL.
`REQ.schedule`	2995	Programa la siguiente revisión de pedidos.
`REQ.poll`	2996	Revisa el estado de los pedidos en SomeDL.
`REQ.announce`	3025	Avisa (y notifica) qué pedidos terminaron.
`REQ.retry`	3040	Vuelve a pedir una canción que falló.
`sysNotify`	3047	Notificación del sistema (Android o escritorio).
`askNotifyPermission`	3053	Pide permiso de notificaciones en el navegador.
`reqRowHTML`	3056	Fila de un pedido de SomeDL.
`reqListHTML`	3065	Lista de pedidos.
`renderReqSoon`	3072	Redibuja la lista de pedidos en un momento.
`V.request`	3076	Pantalla Solicitar canción.
`refreshFromJellyfin`	3136	«Actualizar desde Jellyfin»: olvida cachés, rehace índices y refresca la cola.
`sessionGone`	3159	Confirma si un 401 es de verdad la sesión vencida.
`sessionExpired`	3170	Sesión vencida: detiene todo y pide entrar.
`render`	3178	Dibuja la pantalla actual con transición y manejo de errores.

Reproductor desde la línea 3214 · 53

Función	Línea	Qué hace
`P.pause`	3224	Pausa marcando que la pidió Nanuq.
`saveQueue`	3227	Guarda la cola y actualiza la ventana por adelantado.
`QPOS.save`	3232	Guarda el segundo de la canción para retomarlo.

Función	Línea	Qué hace
QPOS.restore	3238	Dónde retomar la canción al abrir la app.
QPOS.clear	3242	Olvida la posición guardada.
setQueue	3246	Reemplaza la cola y empieza a reproducir.
addToQueue	3247	Agrega canciones al final o a continuación.
ptime	3257	Segundo actual de la canción (aquí o en el otro dispositivo).
pdur	3258	Duración de la canción.
setSrc	3264	Cambia la fuente del audio reiniciando su estado.
seekTo	3268	Salta a un segundo (en conversiones en vivo pide desde ahí).
playIndex	3289	Reproduce la canción N de la cola.
armStallWatch	3319	Avisa si una canción tarda demasiado en empezar.
MP3.add	3333	Recuerda una canción que debe pedirse en MP3.
localFailed	3337	La copia local falló: suena del servidor (y se vuelve a descargar).
canFallback	3349	¿Se puede pedir esta canción en MP3?
fallbackMp3	3352	Pide la canción en MP3 320 kbps si la original no arranca.
armFallback	3362	Programa el paso a MP3 si no arranca en 6 s.
playFailed	3368	Explica por qué no se pudo reproducir.
audioDead	3389	¿El audio quedó muerto (error de red)?
replayHere	3390	Vuelve a pedir la canción desde el segundo en que iba.
fadeStop	3398	Detiene un desvanecido en curso.
fadeTo	3399	Lleva el volumen a un valor con desvanecido.
userVol	3411	Volumen elegido por ti (bajado si habla el DJ).
pauseSoft	3412	Pausa con desvanecido.
playSoft	3417	Reanuda con desvanecido (o recarga si otra app pausó).
partyNow	3426	¿Los controles manejan la fiesta ahora?
partyHostOpen	3429	¿Hay una sala de fiesta abierta en esta computadora?
toggle	3430	Reproducir/pausar (fiesta, control remoto o aquí).
next	3439	Siguiente canción (respeta repetir y el fin de la cola).
prev	3451	Canción anterior (o al inicio si van más de 3 s).
seekFrac	3452	Salta a una fracción de la canción.
removeAt	3453	Quita una canción de la cola.
clearQueue	3458	Vacía la cola.
shuffleQueue	3459	Mezcla la cola (la actual queda primero).

Función	Línea	Qué hace
setMode	3467	Cambia el modo de repetición.
updateModeBtn	3471	Actualiza el botón de repetición.
loadPeaks	3473	Carga la forma de onda de una canción.
cssVar	3482	Lee una variable CSS del tema (en caché).
drawWaves	3483	Dibuja las barras de progreso o la forma de onda.
roundRect	3511	Rectángulo redondeado en canvas.
updateNow	3513	Actualiza todo lo que muestra la canción actual.
updatePlayState	3543	Íconos de reproducir/pausa y estado de los controles del sistema.
DISC.can	3558	¿Hay puente de Discord (escritorio)?
DISC.update	3559	Programa el aviso a Discord.
DISC.send	3560	Manda a Discord lo que suena (o lo quita).
dcStatusPaint	3576	Línea de estado de Discord en Ajustes.
rcKeep	3597	Audio en silencio para que el sistema mantenga los controles.
sessionHold	3611	Mantiene los controles del sistema mientras carga o al terminar la cola.
rcKeepOff	3617	Detiene el audio en silencio.
stallCheck	3657	Detecta audio atorado a media canción y lo vuelve a pedir.
netBack	3666	Volvió la red: reanuda lo cortado y reconecta.
paintVol	3697	Dibuja la barra de volumen y su ícono.

Letras desde la línea 3702 · 32

Función	Línea	Qué hace
lrcKey	3705	Clave de una letra guardada en el dispositivo.
lrcTime	3706	Formatea un tiempo como mm:ss.cc (LRC).
parseLRC	3707	Lee un archivo LRC (marcas de tiempo y offset).
toLRC	3721	Arma un archivo LRC limpio.
cleanLyricsText	3732	Limpia una letra escrita o pegada (controles, HTML, tamaño).
lyricsFromText	3741	Convierte texto (LRC o simple) en letra.
localLyrics	3742	Letra guardada en este dispositivo.
lyrSame	3750	¿Dos letras son iguales (texto y tiempos)?
lyricsSaved	3758	Muestra la letra recién guardada hasta que Jellyfin la sirva.
getLyrics	3773	Letra de una canción (local, pendiente, caché o servidor).
setLyricsMode	3781	Abre o cierra la letra con transición.

Función	Línea	Qué hace
setQueueMode	3794	Abre o cierra la cola del reproductor con transición.
renderSheetQueue	3805	Lista de la cola dentro del reproductor.
applyLyricsMode	3814	Aplica el modo letra sin transición.
reloadLyrics	3822	Vuelve a cargar la letra actual.
renderLyrics	3823	Dibuja la letra de la canción actual.
estimateTimes	3847	Reparte tiempos aproximados para letras sin sincronizar.
syncLyrics	3854	Resalta la línea que suena y desplaza la letra.
refreshLyricsButtons	3890	Botón Letra o Buscar letra según haya letra.
syncStart	3905	Abre el editor para sincronizar a mano.
syncEnd	3919	Cierra el editor.
syncCancel	3923	Cancela la sincronización.
renderSync	3924	Dibuja el editor de sincronización.
syncEditorTick	3931	Marca en el editor la línea que suena.
syncMark	3938	Marca el inicio de la línea resaltada.
syncUndo	3951	Deshace la última marca.
syncNudge	3959	Mueve la última marca ¼ s.
syncSave	3968	Completa los tiempos y guarda el LRC.
saveLyricsText	3985	Guarda una letra en Jellyfin (o en el dispositivo si no se puede).
lyricsSearch	3997	Busca letras en LrcLib y deja elegir una.
lyricsPaste	4027	Diálogo para pegar o editar una letra.
forgetLocalLyrics	4045	Quita la letra guardada en este dispositivo.

Cola desde la línea 4048 · 4

Función	Línea	Qué hace
renderQueue	4050	Dibuja el panel de la cola.
renderQueueAll	4059	Redibuja todas las vistas de la cola.
moveQueue	4064	Mueve una canción de la cola (también en el control remoto).
markCurrent	4116	Resalta la canción que suena en las listas.

Avisos, menús y diálogos desde la línea 4119 · 2

Función	Línea	Qué hace
toast	4121	Aviso breve en pantalla.
menu	4122	Menú de opciones (hoja inferior).

Mis dispositivos desde la línea 4130 · 63

Función	Línea	Qué hace
devLabel	4135	Nombre amable de un dispositivo.
devName	4137	Nombre de un dispositivo para frases.
expand6	4139	Expande una dirección IPv6.
ipKey	4145	Red y privacidad de una dirección IP.
sameNetwork	4153	¿Dos direcciones están en la misma red?
RC.on	4165	¿Mis dispositivos está activado?
RC.start	4166	Arranca Mis dispositivos (WebSocket y búsqueda).
RC.stop	4167	Detiene Mis dispositivos.
RC.connect	4173	Abre el WebSocket de Jellyfin con reintentos.
RC.send	4186	Manda un mensaje por el WebSocket.
RC.onMessage	4187	Atiende los mensajes del WebSocket (órdenes, favoritas, keep-alive).
RC.fromMe	4198	¿La orden viene de tu propio usuario?
RC.onPlaystate	4205	Ejecuta pausar, siguiente, buscar… recibidos.
RC.onPlay	4218	Ejecuta una orden de reproducir (y adopta el DJ si viene con él).
RC.onGeneral	4240	Ejecuta repetir o mezclar recibidos.
RC.queueForReport	4247	Cola que se informa a Jellyfin.
RC.reportSoon	4248	Informa el estado a Jellyfin en un momento.
RC.schedule	4254	Programa la siguiente búsqueda de dispositivos.
RC.poll	4255	Busca tus otros Nanuq y sigue al que controlas.
RC.maybeFollow	4278	Se conecta solo a otro Nanuq que suena en tu red.
RC.enter	4284	Empieza a controlar otro dispositivo.
RC.leave	4300	Deja de controlar y recupera tu cola.
RC.apply	4308	Refleja aquí lo que suena en el otro dispositivo.
RC.pos	4339	Segundo estimado de la canción en el otro dispositivo.
RC.tick	4340	Actualiza barra, letra y controles del sistema en control remoto.
RC.decorate	4347	Indica «Sonando en …» en el reproductor.
RC.fail	4353	Avisa que una orden no se pudo enviar.
RC.cmd	4354	Manda una orden al dispositivo que suena.
RC.seek	4360	Salta en el otro dispositivo.
RC.window	4361	Ventana de la cola que se manda (≤ 200).
RC.playQueue	4362	Manda una cola para reproducir allá.

Función	Línea	Qué hace
RC.add	4371	Agrega canciones a la cola de allá.
RC.reorder	4381	Manda la cola reordenada (conservando el DJ).
RC.general	4392	Manda una orden general allá.
RC.here	4394	«Reproducir aquí»: trae la música (y el DJ) a este dispositivo.
RC.there	4417	Pasa la música (y el DJ) a otro dispositivo.
__nanuqTick	4436	Latido cada 20 s desde Android (WebSocket, fiesta, audio atorado).
__nanuqVis	4444	Android avisa si la app está a la vista.
openDevices	4445	Menú Mis dispositivos.
updateFavBtn	4476	Botón de favorita del reproductor.
setFavLocal	4482	Marca la favorita en la cola local.
favPending	4489	Recuerda un cambio de favorita reciente.
favMerge	4490	Aplica los cambios recientes sobre la lista del servidor.
favHold	4508	Protege la favorita de la canción que suena (fallo de Jellyfin 12).
favHeld	4511	¿Se está protegiendo esta favorita?
favReassert	4512	Vuelve a guardar la favorita después de cada aviso de reproducción.
favRemote	4520	Atiende cambios de favoritas hechos en otro lado.
refreshFav	4542	Confirma con el servidor si la canción es favorita.
toggleFav	4553	Marca o quita la favorita y lo comprueba.
favConfirm	4573	Segunda confirmación contra la sección de favoritas.
updateDlBtn	4587	Botón Descargar del reproductor.
dlAddToast	4596	Aviso al pedir una descarga.
dlTap	4605	Toque en Descargar (pedir o administrar).
connTest	4618	«Probar conexión»: mide API, carátula, audio, saltos y reproductor.
startRadio	4746	Radio a partir de una canción.
addToPlaylistMenu	4760	Menú para agregar a una playlist (o crear una).
songOptions	4775	Opciones comunes de una canción.
queueMenu	4782	Menú de una canción de la cola.
trackMenu	4795	Menú de una canción de una lista.
askConfirm	4803	Diálogo de confirmación.
askName	4809	Diálogo para escribir un nombre.
openSheet	4817	Abre el reproductor maximizado.

Función	Línea	Qué hace
closeSheet	4825	Cierra el reproductor maximizado.

Fiesta (watch party) desde la línea 4828 · 65

Función	Línea	Qué hace
partyHost	4835	¿Este dispositivo puede ser anfitrión (escritorio)?
partyGuest	4836	¿Este dispositivo es invitado (Android)?
partyOn	4837	¿Hay fiesta en este dispositivo?
partySongUrl	4839	URL en MP3 320 kbps para mandar una canción a la fiesta.
partyCover	4845	Miniatura JPEG de 240 px de la carátula.
partyCoverOk	4855	¿La carátula viene de la sala (127.0.0.1 con token)?
partyClean	4858	Limpia el estado que manda la computadora (no confiable).
stopForParty	4870	Detiene tu música al abrir la sala.
PH.active	4885	¿La fiesta maneja los controles?
PH.init	4886	Prepara el reproductor de la fiesta y escucha a main.js.
PH.onEv	4898	Atiende los eventos de la sala (estado, llegó, salió).
PH.apply	4904	Aplica el estado de la sala (modo fiesta de la computadora).
PH.list	4916	Lista de canciones de la sala.
PH.open	4917	Abre la sala.
PH.close	4923	Cierra la sala.
PH.jump	4925	Pone ya una canción de «A continuación».
PH.order	4931	Cambia el orden de la lista.
PH.next	4933	Sigue la lista: pone la siguiente que ya llegó.
PH.play	4940	Reproduce una canción de la fiesta.
PH.stopCur	4948	Detiene la canción actual de la fiesta.
PH.done	4956	Terminó una canción: la borra y pasa a la siguiente.
PH.toggle	4957	Pausa o reanuda la fiesta.
PH.yieldTo	4970	Pusiste tu música: la fiesta se pausa y cede los controles.
PH.media	4977	Controles del sistema con la canción de la fiesta.
PH.report	4989	Avisa a la sala qué suena.
PH.loadLyrics	4994	Carga la letra de la canción de la fiesta.
PH.line	5004	Línea de letra actual y la siguiente.
PH.tick	5012	Karaoke: cada segundo, tiempo y línea a los teléfonos.
PG.connect	5022	Entra a una sala (con código o llave) y reconecta solo.
PG.leave	5049	Sale de la fiesta.

Función	Línea	Qué hace
PG.joined	5059	¿Estás en una fiesta?
PG.ui	5060	Modo fiesta del teléfono (oculta barras y reproductor).
PG.wake	5069	El teléfono despertó: vuelve a entrar si hace falta.
PG.resume	5075	Al abrir la app, vuelve a la fiesta en curso.
PG.abortUp	5080	Cancela la subida en curso.
PG.onMsg	5081	Atiende los mensajes de la sala.
PG.add	5108	Manda una canción a la fiesta (letra, carátula y audio por partes).
PG.remove	5164	Cancela una canción que todavía llega.
PG.find	5166	Busca salas en la red (UDP desde Android).
partyAddr	5186	Convierte la dirección escrita en ws://.../fiesta.
iglooHTML	5197	HTML del iglú.
iglooBear	5213	Nanuq de un invitado con el disfraz de su tema.
iglooHoldSVG	5220	Carátula en alto con destellos (quien escogió la canción).
iglooHeldOnTop	5231	Pone encima de la carátula el objeto que traía el Nanuq.
partyGuestColor	5242	Color de la etiqueta según el tema del invitado.
partyCoverSrc	5250	URL de la carátula de una canción de la fiesta.
iglooSync	5258	Coloca a los invitados en el círculo sin redibujar.
iglooLive	5292	Anima el iglú solo mientras se ve.
iglooQueue	5297	Fila «A continuación» a pantalla completa.
iglooQueueEdge	5309	Degradado si hay más canciones fuera de la vista.
partyVolumeStep	5385	Volumen con ↑ ↓ a pantalla completa y su aviso.
iglooIdle	5398	Esconde el puntero tras 2.5 s quieto.
iglooVinyl	5408	Funda y disco de vinilo de la canción que suena.
iglooLine	5420	Letra en el techo con destello.
renderPartySoon	5427	Redibuja la fiesta en un momento.
partyPaintTick	5431	Actualiza el karaoke (línea, siguiente, tiempo).
partyPaintUp	5437	Progreso de la canción que mandas.
partyRowsHTML	5438	Lista de canciones de la fiesta.
partyGuestsHTML	5447	Lista de invitados.
partyNowHTML	5448	Lo que suena en la fiesta.
paintParty	5453	Dibuja la pantalla de la fiesta (anfitrión o invitado).
partySongsHTML	5514	Canciones para mandar.

Función	Línea	Qué hace
partySearch	5515	Buscador de la fiesta (playlists, artistas, álbumes, canciones, letras).
partyBrowse	5543	Canciones de un artista, álbum o playlist en la fiesta.
V.party	5558	Pantalla Fiesta.

Acciones globales desde la línea 5572 · 1

Función	Línea	Qué hace
d0	5574	Atajo del dataset de un elemento.

Entrar desde la línea 5732 · 11

Función	Línea	Qué hace
recentServers	5735	Servidores usados antes.
rememberServer	5736	Recuerda un servidor.
forgetServer	5742	Olvida un servidor.
probeTimed	5744	Prueba un servidor midiendo cuánto tarda.
canDiscover	5755	¿Se pueden buscar servidores en la red?
discoverServers	5757	Busca Jellyfin en la red (UDP 7359).
serverList	5776	Lista de servidores con su estado.
cleartextRisk	5803	¿La contraseña viajaría sin cifrar por internet?
showLogin	5814	Pantalla de entrar.
startApp	5880	Arranca la app con la sesión (cola, posición, DJ, Inicio).
logout	5896	Cierra sesión y borra lo personal.

17 Resto de la página

Idiomas y service worker.



web/i18n.js

995 líneas · 1 funciones

Función	Línea	Qué hace
tx	984	Traduce un texto en español al idioma elegido y pone los valores {0}, {1}...

web/sw.js

45 líneas · 3 funciones

Función	Línea	Qué hace
install	8	Guarda en caché los archivos de la app al instalar el service worker.
activate	12	Borra las cachés de versiones anteriores.
fetch	16	Sirve solo los archivos de la app (red primero para HTML/JS/CSS); la API y la música van directo a la red.

18 Android

Java nativo y el puente de JavaScript.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 líneas · 3 funciones

Función	Línea	Qué hace
Discovery.Discovery	27	Constructor privado (solo métodos estáticos).
Discovery.find	30	Busca por UDP (Jellyfin o fiestas) y devuelve las respuestas en JSON.
Discovery.send	72	Manda la pregunta a todas las direcciones de difusión.

android/app/src/main/java/com/nanuq/app/Downloads.java

591 líneas · 44 funciones

Función	Línea	Qué hace
Downloads.get	43	Instancia única de las descargas.
Downloads.Downloads	72	Prepara carpeta, índice, servidor local y vigilancia de red.
Downloads.audio	83	Archivo de audio para el servidor local.
Downloads.audioMime	84	Tipo del audio para el servidor local.
Downloads.cover	85	Carátula para el servidor local.
Downloads.prefetch	86	Canción guardada por adelantado para el servidor local.
Downloads.prefetchMime	87	Tipo de la canción guardada por adelantado.
Downloads.onAvailable	91	Volvió la red: reintenta las descargas.
Downloads.onCapabilitiesChanged	92	Cambió la red: revisa la cola de descargas.
Downloads.setListener	97	Conecta la página para recibir cambios.
Downloads.configure	101	Máximo de canciones y solo Wi-Fi.
Downloads.base	117	Dirección del servidor local.
Downloads.okUrl	127	¿La URL tiene la forma de audio o carátula de Jellyfin?
Downloads.openSameHost	141	Abre una conexión siguiendo redirecciones solo al mismo servidor.
Downloads.enqueue	162	Agrega una descarga (o la actualiza).
Downloads.setScore	196	Actualiza cuántas veces sonó una descarga.
Downloads.remove	202	Quita una descarga.
Downloads.removeAll	208	Quita todas (o solo las automáticas).
Downloads.list	214	Lista de descargas en JSON.

Función	Línea	Qué hace
`Downloads.network`	222	Red actual (wifi, datos, ninguna), mirando debajo de una VPN.
`Downloads.underlying`	236	Red real debajo de una VPN.
`Downloads.file`	245	Archivo de una descarga lista.
`Downloads.coverFile`	253	Carátula de una descarga.
`Downloads.setCover`	262	Pide la carátula de una descarga vieja.
`Downloads.fetchCoverLater`	270	Baja la carátula en segundo plano.
`Downloads.fetchCover`	273	Baja la carátula (≤ 15 MB).
`Downloads.mime`	301	Tipo de audio de una descarga.
`Downloads.retryAndKick`	309	Reintenta las fallidas y sigue.
`Downloads.kick`	316	Arranca la cola de descargas.
`Downloads.kickLocked`	318	Arranca la cola (ya con el candado).
`Downloads.loop`	324	Baja las pendientes de una en una.
`Downloads.nextPending`	344	La siguiente (primero las pedidas por ti).
`Downloads.download`	354	Baja una canción con topes y espacio libre.
`Downloads.victim`	427	La automática menos escuchada (para hacer lugar).
`Downloads.removeEntry`	436	Borra una descarga y sus archivos.
`Downloads.caps`	450	Capacidades de la red activa.
`Downloads.unmetered`	455	¿La red es sin medición (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	460	¿Se puede descargar con esta red?
`Downloads.safe`	468	Nombre de archivo seguro para un id.
`Downloads.toJson`	470	Descarga en JSON para la página.
`Downloads.emit`	488	Avisa a la página un cambio.
`Downloads.ensureLoaded`	494	Lee el índice cuando el almacén se abre.
`Downloads.load`	496	Lee el índice cifrado (o recupera desde los archivos).
`Downloads.save`	562	Guarda el índice cifrado.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 líneas · 7 funciones

Función	Línea	Qué hace
`LocalServer.LocalServer`	46	Abre el servidor en 127.0.0.1 con token aleatorio.
`LocalServer.base`	60	Dirección base con el token.
`LocalServer.acceptLoop`	62	Acepta conexiones (máx. 32 a la vez).
`LocalServer.readLine`	74	Lee una línea de la petición con tope y hora límite.
`LocalServer.handle`	86	Atiende una petición (10 s y ≤ 100 encabezados).
`LocalServer.status`	129	Responde un código de estado.

Función	Línea	Qué hace
`LocalServer.serveFile`	135	Entrega un archivo con rangos (saltar dentro de la canción).

android/app/src/main/java/com/nanuq/app/MainActivity.java

406 líneas · 44 funciones

Función	Línea	Qué hace
`MainActivity.run`	63	Latido cada 20 s hacia la página.
`MainActivity.onCreate`	70	Crea el WebView, conecta servicios y carga la app.
`MainActivity.createWeb`	88	Configura el WebView (sin acceso a archivos, puente).
`MainActivity.recreateWeb`	114	Rehace el WebView si el sistema lo cerró.
`MainActivity.dispatchMedia`	124	Pasa un botón de la notificación a la página.
`MainActivity.onBackPressed`	135	Atrás: la página decide o la app va al fondo.
`MainActivity.onStart`	144	La app volvió a la vista.
`MainActivity.onStop`	147	La app pasó al fondo (guarda el almacén).
`MainActivity.onDestroy`	150	Libera el WebView y la notificación.
`MainActivity.setBarColor`	164	Color de las barras del sistema según el tema.
`MainActivity.showNotice`	185	Notificación de «canción solicitada lista».
`MainActivity.setMetadata`	213	Puente: título, artista, álbum y carátula.
`MainActivity.setPlaybackState`	214	Puente: reproduciendo o en pausa.
`MainActivity.setActions`	215	Puente: botones disponibles.
`MainActivity.setPosition`	216	Puente: posición y duración.
`MainActivity.dlList`	224	Puente: lista de descargas.
`MainActivity.dlEnqueue`	225	Puente: descargar una canción.
`MainActivity.dlCover`	226	Puente: pedir una carátula.
`MainActivity.dlBase`	227	Puente: dirección del servidor local.
`MainActivity.dlRemove`	228	Puente: quitar una descarga.
`MainActivity.dlRemoveAll`	229	Puente: quitar descargas.
`MainActivity.dlConfig`	230	Puente: ajustes de descargas.
`MainActivity.dlScore`	231	Puente: reproducciones de una descarga.
`MainActivity.dlNetwork`	232	Puente: tipo de red.
`MainActivity.pfSet`	235	Puente: ventana de lo que sigue por adelantado.
`MainActivity.pfReady`	236	Puente: ¿ya está guardada esa canción?
`MainActivity.pfBroken`	237	Puente: esa canción guardada no sonó.
`MainActivity.http`	240	Puente: petición a SomeDL sin CORS.
`MainActivity.secKind`	248	Puente: estado del almacén cifrado.
`MainActivity.secGet`	249	Puente: leer un dato cifrado.

Función	Línea	Qué hace
`MainActivity.secSet`	250	Puente: guardar un dato cifrado.
`MainActivity.secDel`	251	Puente: borrar un dato cifrado.
`MainActivity.secReset`	253	Puente: empezar de cero.
`MainActivity.videoBegin`	257	Puente: empieza a guardar un video (MP4/WebM).
`MainActivity.videoChunk`	269	Puente: un trozo del video (≤ 300 MB).
`MainActivity.videoEnd`	281	Puente: termina y guarda en Películas › Nanuq.
`MainActivity.videoShare`	323	Puente: compartir el video recién guardado.
`MainActivity.notice`	332	Puente: notificación del sistema.
`MainActivity.partyDiscover`	335	Puente: buscar fiestas por UDP.
`MainActivity.discover`	343	Puente: buscar Jellyfin por UDP.
`MainActivity.onRenderProcessGone`	354	El proceso del WebView murió: se rehace.
`MainActivity.shouldInterceptRequest`	360	Sirve la app desde los assets en http://localhost.
`MainActivity.shouldOverrideUrlLoading`	377	Solo enlaces http(s) salen al navegador.
`MainActivity.mime`	399	Tipo de un archivo de los assets.

`android/app/src/main/java/com/nanuq/app/MediaCenter.java` 372 líneas · 33 funciones

Función	Línea	Qué hace
`MediaCenter.get`	52	Instancia única del centro multimedia.
`MediaCenter.MediaCenter`	72	Crea la MediaSession y el canal de notificación.
`MediaCenter.onPlay`	83	Botón reproducir del sistema.
`MediaCenter.onPause`	84	Botón pausa del sistema.
`MediaCenter.onStop`	85	Botón detener del sistema.
`MediaCenter.onSkipToNext`	86	Botón siguiente del sistema.
`MediaCenter.onSkipToPrevious`	87	Botón anterior del sistema.
`MediaCenter.onSeekTo`	88	Saltar desde el sistema.
`MediaCenter.onFastForward`	89	Adelantar desde el sistema.
`MediaCenter.onRewind`	90	Retroceder desde el sistema.
`MediaCenter.attach`	96	Conecta la página.
`MediaCenter.detach`	98	Desconecta la página.
`MediaCenter.hasPage`	101	¿Hay página que reciba órdenes?
`MediaCenter.stopIfNoPage`	104	Sin página, quita la notificación.
`MediaCenter.dispatch`	113	Manda una acción a la página.
`MediaCenter.setMetadata`	120	Datos de la canción (y carga la carátula).
`MediaCenter.setPlaybackState`	141	Estado de reproducción.

Función	Línea	Qué hace
`MediaCenter.setActions`	152	Botones disponibles.
`MediaCenter.setPosition`	160	Posición y duración.
`MediaCenter.isPlaying`	174	¿Está sonando?
`MediaCenter.currentPosition`	176	Posición actual estimada.
`MediaCenter.publishMetadata`	181	Publica los datos en la MediaSession.
`MediaCenter.publishState`	191	Publica el estado y los botones.
`MediaCenter.refresh`	212	Ajusta servicio, notificación y bloqueos.
`MediaCenter.shutdown`	240	Detiene todo al cerrar la app.
`MediaCenter.buildNotification`	251	Arma la notificación de reproducción.
`MediaCenter.action`	280	Un botón de la notificación.
`MediaCenter.serviceIntent`	284	Intent de un botón hacia el servicio.
`MediaCenter.contentIntent`	292	Abrir la app desde la notificación.
`MediaCenter.onReceive`	300	Audífonos desconectados: pausa.
`MediaCenter.setNoisyReceiver`	305	Escucha la desconexión de audífonos.
`MediaCenter.loadBitmap`	319	Carga la carátula con topes (8 MB, reducida).
`MediaCenter.readCapped`	364	Lee con tope de bytes.

`android/app/src/main/java/com/nanuq/app/NativeHttp.java` 84 líneas · 5 funciones

Función	Línea	Qué hace
`NativeHttp.NativeHttp`	19	Constructor privado (solo métodos estáticos).
`NativeHttp.Result`	24	Respuesta (estado y texto).
`NativeHttp.allowed`	30	¿Ruta permitida de SomeDL?
`NativeHttp.fetch`	37	Petición sin CORS solo a SomeDL, sin redirecciones.
`NativeHttp.readAll`	71	Lee la respuesta (≤ 4 MB).

`android/app/src/main/java/com/nanuq/app/PlaybackService.java` 101 líneas · 7 funciones

Función	Línea	Qué hace
`PlaybackService.onCreate`	31	Prepara los bloqueos de CPU y Wi-Fi.
`PlaybackService.onStartCommand`	46	Primer plano y botones de la notificación.
`PlaybackService.setLocks`	65	Toma o suelta los bloqueos.
`PlaybackService.release`	76	Sale del primer plano y termina.
`PlaybackService.onTaskRemoved`	84	Quitada de recientes: cierra todo.
`PlaybackService.onDestroy`	92	Suelta los bloqueos.
`PlaybackService.onBind`	99	Sin enlace (servicio iniciado).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 líneas · 16 funciones

Función	Línea	Qué hace
Prefetch.get	40	Instancia única de la caché por adelantado.
Prefetch.Prefetch	61	Vacía la caché anterior y vigila la red.
Prefetch.onAvailable	71	Volvió la red: sigue bajando.
Prefetch.set	80	Recibe la ventana deseada y borra lo que sobra.
Prefetch.broken	109	Una guardada no sonó: se borra y no se repite.
Prefetch.ready	116	¿Está lista esa canción?
Prefetch.file	118	Archivo de una canción lista.
Prefetch.mime	125	Tipo de audio.
Prefetch.drop	130	Quita una canción y su archivo.
Prefetch.deleteFiles	132	Borra los archivos de una canción.
Prefetch.kick	139	Arranca las descargas.
Prefetch.kickLocked	141	Arranca (ya con el candado).
Prefetch.loop	147	Baja las pendientes de una en una (respeta el tope total).
Prefetch.online	160	¿Hay red?
Prefetch.fetch	166	Baja una canción con topes.
Prefetch.trim	211	Si se pasa del tope total, suelta las más lejanas.

android/app/src/main/java/com/nanuq/app/SecureStore.java 192 líneas · 16 funciones

Función	Línea	Qué hace
SecureStore.get	44	Instancia única (o lee un valor si recibe una clave).
SecureStore.SecureStore	58	Abre el almacén al crearse.
SecureStore.open	69	Abre clave (Keystore) y archivo: listo, locked o lost.
SecureStore.kind	104	Estado (reintenta si estaba locked).
SecureStore.reset	113	Empezar de cero: aparta el archivo y crea clave nueva.
SecureStore.ready	122	¿Está abierto?
SecureStore.set	129	Guarda un valor (≤ 8 MB).
SecureStore.del	136	Borra un valor.
SecureStore.validKey	142	¿Clave válida?
SecureStore.scheduleSave	144	Guarda en 400 ms.
SecureStore.flush	150	Guarda ya.
SecureStore.saveNow	152	Cifra y escribe el archivo.
SecureStore.encrypt	161	Cifra con AES-256-GCM.
SecureStore.decrypt	171	Descifra y verifica.

Función	Línea	Qué hace
`SecureStore.readAll`	178	Lee un archivo (≤ 64 MB).
`SecureStore.writeAtomic`	186	Escribe de forma atómica.

`android/bridge/android-bridge.js` 97 líneas · 9 funciones

Función	Línea	Qué hace
`abs`	17	Convierte una URL relativa en absoluta.
`MediaMetadata`	19	Reemplazo de MediaMetadata para la página.
`bestArt`	28	Elige la carátula más adecuada (≤ 512 px) para la notificación.
`mediaSession.metadata`	42	Manda título, artista, álbum y carátula a Android.
`mediaSession.playbackState`	47	Manda el estado de reproducción a Android.
`mediaSession.setActionHandler`	54	Registra los botones que maneja la página.
`mediaSession.setPositionState`	58	Manda la posición solo cuando se desvía.
`__nanuqMedia`	76	Ejecuta en la página un botón de la notificación o de los audífonos.
`sendTheme`	87	Manda el color del tema a las barras del sistema.

`android/build-apk.sh` 19 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	1	Compila el APK y lo deja en dist/<versión>/.

19 Escritorio

Electron y su empaquetado.



linux/main.js

793 líneas · 56 funciones

Función	Línea	Qué hace
serve	43	Servidor interno 127.0.0.1:47863 que entrega la app a la ventana (y la fiesta), endurecido.
broadcastOf	75	Dirección de difusión de una red IPv4.
discover	76	Busca servidores Jellyfin por UDP 7359.
fromApp	96	¿El mensaje IPC viene de la página de la app?
vaultBackend	128	Qué llavero del sistema hay (GNOME Keyring, KWallet, DPAPI, Llavero).
secFile	144	Ruta del archivo de datos cifrado.
secKeyFile	145	Ruta de la clave de datos cifrada con el llavero.
writeAtomic	146	Escribe un archivo de forma atómica (0600).
secInit	149	Abre el almacén cifrado (AES-256-GCM): locked, lost o listo.
secFlush	186	Escribe el almacén cifrado en el disco.
dcSet	224	Anota el estado de la conexión con Discord.
dcPaths	225	Sockets o tuberías de Discord posibles (solo los tuyos).
dcWrite	236	Manda un mensaje con el formato de Discord.
dcRetry	237	Vuelve a buscar Discord en 10 s.
dcConnect	238	Se conecta a Discord probando cada socket y saluda.
dcData	254	Lee los mensajes de Discord con topes (READY, ERROR, cierre, ping).
dcFlush	277	Manda la actividad respetando el ritmo de Discord (y la mínima si la rechazó).
dcActivity	288	Arma la actividad «Escuchando» limpia y acotada.
jparse	325	Lee JSON de afuera sin toString/valueOf/__proto__.
ptStr	326	Convierte a texto seguro y acotado (sin controles).
ptIp	327	Dirección IP sin el prefijo ::ffff:.
ptLocal	331	¿La dirección es de tu red local?
ptOpenNow	338	¿La sala está abierta?

Función	Línea	Qué hace
ptEmit	339	Avisa a la página un evento de la sala.
ptCleanTemp	341	Borra carpetas temporales de salas anteriores.
ptSniff	346	¿Es audio? (por sus primeros bytes).
ptCover	359	Valida la carátula que manda un teléfono (JPEG/PNG/WebP ≤ 150 KB).
ptPublic	368	Datos públicos de una canción de la lista.
ptGuests	372	Invitados presentes y ausentes.
ptAddrs	373	Direcciones IPv4 de esta computadora.
ptStatus	378	Estado completo de la sala para la página.
ptState	384	Avisa el estado a todos (agrupado en 150 ms).
ptStateNow	385	Manda el estado ya.
ptKill	391	Quita una canción de la lista y del disco.
ptFrame	400	Escribe un marco WebSocket.
ptSend	407	Manda un mensaje JSON a un teléfono.
ptDrop	408	Cierra la conexión de un teléfono (con motivo).
ptGone	416	Un teléfono se desconectó (queda ausente o salió).
ptUpgrade	427	Acepta la conexión WebSocket (solo red local, topes).
ptData	447	Lee marcos WebSocket con todas las validaciones.
ptMsg	480	Atiende un mensaje de un teléfono.
ptNewCode	493	Cambia el código de la sala.
ptHello	497	Entrada de un teléfono: código, llave, fuerza bruta.
ptAdd	527	Un teléfono anuncia una canción (con topes).
ptChunk	543	Recibe un pedazo de la canción y lo escribe al disco.
ptEnd	558	La canción terminó de llegar: comprueba que sea audio.
ptServe	573	Sirve canciones y carátulas de la sala a la ventana (con Range).
ptGuestCover	600	Sirve carátulas a los teléfonos (solo con su llave).
ptGc	609	Limpieza cada 30 s (viejas, a medias, ausentes, mudas).
ptListen	619	Abre el puerto de la sala (47864 o uno libre).
ptOpen	632	Abre la sala: servidor, UDP, código y token.
ptClose	659	Cierra la sala y corta todas las conexiones.
stateFile	719	Archivo con el tamaño y posición de la ventana.
loadState	720	Lee el tamaño y posición de la ventana.
saveState	721	Guarda el tamaño y posición de la ventana.

Función	Línea	Qué hace
`start`	727	Arranca la app: almacén, servidor, ventana, permisos y protecciones.

`linux/preload.js` 25 líneas · 6 funciones

Función	Línea	Qué hace
`nanuqDesktop.discover`	7	Puente: buscar Jellyfin en la red.
`nanuqDesktop.http`	8	Puente: petición sin CORS a SomeDL.
`nanuqDesktop.discord`	11	Puente: actividad de Discord (o su estado).
`nanuqDesktop.party.op`	14	Puente: operaciones de la fiesta (abrir, cerrar, saltar, ordenar, karaoke…).
`nanuqDesktop.party.on`	15	Puente: escuchar los eventos de la sala.
`nanuqDesktop.secure.*`	18	Puente síncrono al almacén cifrado (kind, get, set, del, reset).

`linux/prepare-web.js` 16 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	3	Copia web/ a www/ y toma la versión de app.js.

`linux/collect.js` 15 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	3	Mueve los paquetes compilados a dist/<versión>/.

`linux/flatpak/build-flatpak.sh` 54 líneas · 1 funciones

Función	Línea	Qué hace
`(script)`	1	Arma el Flatpak, lo publica en el repositorio local firmado y lo instala o actualiza.

20 Compilación

El script que compila todo.



scripts/build.sh

68 líneas · 1 funciones

Función	Línea	Qué hace
(script)	1	Sube la versión y compila Android, Linux, Flatpak y Windows.

Hecho con cariño (y mucho hielo)

Si cambias algo, recuerda: la interfaz vive en web/ y cada texto pasa por tx().

